Source Code and Output

Problem 1: Stack as Linked List

Code:

```
#include<iostream>
using namespace std;

class Stack{
    struct node{
        int data;
        node* next;
        node(int val): data(val), next(nullptr){}
        void display(){
            cout << "At address: " << this << endl;
            cout << "Data: " << data << endl;
            cout << "Next Address: " << next << endl;
            cout <<
"~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"
<< endl;
        }
    };

    public:
        node* top;
        int stacksize;
        Stack() : top(nullptr), stacksize(0){}
        int push(int data){
            node* new_node = new node(data);
            if (new_node == nullptr) return -1;
            new_node->next = top;
            top = new_node;
            stacksize++;

            cout << "Successfully pushed a new node at top! The new node
pushed is:" << endl;
            new_node->display();

            return 0;
        }
        int pop(){
            if (top==nullptr) {
                cout << ("Underflow!")<< endl;
                return -1;
            }
            int value = top->data;
            node* temp = top;
            top = top-> next;

            cout << "Successfully deleted a new node at top! The node
```

```
deleted is:" << endl;
            temp->display();

            delete temp;
            stacksize--;
            return value;
        }

        int peek_top(){
            if (top==nullptr) {
                cout << ("Underflow!")<< endl;
                return -1;
            }
            else return top->data;
        }

};

int main(){
    Stack myStack;
    myStack.push(10);
    myStack.push(20);
    myStack.pop();
    myStack.push(30);
    myStack.push(40);
    myStack.pop();

    myStack.peek_top();

    cout << "Current Queue Size: " << myStack.stacksize << endl;
    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 05_DoublyLinkedList % ./01_StackasLL
Successfully pushed a new node at top! The new node pushed is:
At address: 0x100f0dd40
Data: 10
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully pushed a new node at top! The new node pushed is:
At address: 0x100f0dd50
Data: 20
Next Address: 0x100f0dd40
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully deleted a new node at top! The node deleted is:
At address: 0x100f0dd50
Data: 20
Next Address: 0x100f0dd40
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
Successfully pushed a new node at top! The new node pushed is:
At address: 0x100f0dd50
Data: 30
Next Address: 0x100f0dd40
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully pushed a new node at top! The new node pushed is:
At address: 0x100f0dd60
Data: 40
Next Address: 0x100f0dd50
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully deleted a new node at top! The node deleted is:
At address: 0x100f0dd60
Data: 40
Next Address: 0x100f0dd50
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Discussions

- We created a Stack Class, providing methods for Push, Pop and Peek.
- Internally the class uses a linked list to store each element.
- The specific operations "Insertion from End" and "Deletion from End" are the only ones needed for LIFO style operations of a Stack.
- To test our implementation, we first pushed few elements and then poped them sequentially.

Problem 2: Queue as Linked List

Code:

```
#include<iostream>
using namespace std;

class Queue{
    struct node{
        int data;
        node* next;
        node(int val): data(val), next(nullptr){}
        void display(){
            cout << "At address: " << this << endl;
            cout << "Data: " << data << endl;
            cout << "Next Address: " << next << endl;
            cout <<
"~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"
<< endl;
        }
    };

    public:
        node* front;
```

```
        node* rear;
        int queuesize;
        Queue() : front(nullptr), rear(nullptr), queuesize(0){}

        int Enqueue(int data){
            node* new_node = new node(data);
            if (new_node == nullptr) return -1;
            if (rear!=nullptr){
                rear->next = new_node;
                rear = new_node;
            }
            else {
                rear = new_node;
                front = new_node;
            }
            queuesize++;
            cout << "Successfully enqueued a new node at rear! The new node
enqueued is:" << endl;
            new_node->display();

            return 0;
        }
        int Dequeue(){
            if (front==nullptr) {
                cout << ("Underflow!")<< endl;
                return -1;
            }
            int value = front->data;
            node* temp = front;
            front = front-> next;

            cout << "Successfully dequeued a node at front! The node
dequeued is:" << endl;
            temp->display();

            delete temp;
            queuesize--;
            return value;
        }
        void traverse_display(){
            cout << "Current List Status:" << endl;
            node* cptr = front;
            if (front==nullptr){
                cout << "|null | null|" << endl;
            }
            else{
                while (cptr != nullptr){
                    if (cptr->next == nullptr){
                        cout << "|" << cptr->data << "|" << "null" << "|";
                    }
                    else{
                        cout << "|" << cptr->data << "|" << cptr->next <<
"|" << "------->" ;
                    }
```

```
                    cptr = cptr->next;
                }
                cout << endl;
            }
            cout <<
"_____________________________________________________________________________"
<< endl;
            cout << endl;
        }
};

int main(){
    Queue myQueue;
    myQueue.traverse_display();
    myQueue.Enqueue(10);
    myQueue.Enqueue(20);
    myQueue.traverse_display();
    myQueue.Enqueue(30);
    myQueue.Dequeue();
    myQueue.traverse_display();
    myQueue.Dequeue();
    myQueue.traverse_display();

    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 05_DoublyLinkedList % ./02_QueueasLL
Current List Status:
|null | null|
_____________________________________________________________________________

Successfully enqueued a new node at rear! The new node enqueued is:
At address: 0x100ce1c80
Data: 10
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully enqueued a new node at rear! The new node enqueued is:
At address: 0x100ce1c90
Data: 20
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|10|0x100ce1c90|------->|20|null|
_____________________________________________________________________________

Successfully enqueued a new node at rear! The new node enqueued is:
At address: 0x100ce1ca0
Data: 30
Next Address: 0
```

```
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Successfully dequeued a node at front! The node dequeued is:
At address: 0x100ce1c80
Data: 10
Next Address: 0x100ce1c90
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|20|0x100ce1ca0|------->|30|null|
________________________________________________________________________________

Successfully dequeued a node at front! The node dequeued is:
At address: 0x100ce1c90
Data: 20
Next Address: 0x100ce1ca0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|30|null|
________________________________________________________________________________
```

Discussions

```
- We created a Queue Class, providing methods for Enqueue and Dequeue.
- Internally the class uses a linked list to store each element.
- The specific operations "Insertion from End" and "Deletion from
beginning" are the only ones needed for FIFO style operations of a Queue.
- To test our implementation, we first enqueued few elements and then
dequeued them sequentially.
```

Problem 3: Linked List representation of Polynomials and Addition

Code:

```cpp
#include <iostream>
using namespace std;

class PolynomialLL{
    public:
        struct node{
            int coeff;
            int pow;
            node* next;
            node(int _coeff, int _pow): coeff(_coeff), pow(_pow),
next(nullptr){}
        };
        node *start;
        PolynomialLL() : start(nullptr){}

        void traverse_display(){
            cout << "Current List Status:" << endl;
```

```
            node* cptr = start;
            if (start==nullptr){
                cout << "|null | null|" << endl;
            }
            else{
                while (cptr != nullptr){
                    if (cptr->next == nullptr){
                        cout << "|" << cptr->coeff << "|" << cptr->pow <<
"|"  << "null" << "|";
                    }
                    else{
                        cout << "|" << cptr->coeff << "|" << cptr->pow <<
"|" << cptr->next << "|" << "-------->" ;
                    }
                    cptr = cptr->next;
                }
                cout << endl;
            }
            cout <<
"_____________________________________________________________________________"
<< endl;
            cout << endl;
        }

        int insert(int _coeff, int _pow){
            node* new_node = new node(_coeff, _pow);
            if (start == nullptr) start=new_node;
            else {
                node* ptr = start;
                node* preptr = start;
                while (ptr->pow >= _pow){
                    preptr = ptr;
                    ptr = ptr->next;
                    if (ptr == nullptr){
                        preptr->next = new_node;
                        ptr = new_node;
                        return -1;
                    }
                    if (ptr->pow == _pow){
                        ptr->coeff += _coeff;
                        delete new_node;
                        return -1;
                    }
                }
                if (ptr==start){
                    new_node -> next = start;
                    start = new_node;
                }
                else{
                    new_node->next = ptr;
                    preptr->next = new_node;
                }
            }
            return 0;
```

```
        }

        PolynomialLL add_polynomials(PolynomialLL other_pol){
            PolynomialLL result;
            node* self = start;
            node* other = other_pol.start;
            if (self==nullptr) result = other_pol;
            else if (other==nullptr) result = *this;
            int i = 0;
            while (self!=nullptr or other != nullptr){
                if (self == nullptr){
                    result.insert(other->coeff, other->pow);
                    other = other->next;
                }
                else if (other == nullptr){
                    result.insert(self->coeff, self->pow);
                    self = self->next;
                }
                else if (self->pow == other-> pow){
                    result.insert(self->coeff + other->coeff, self->pow);
                    self = self->next;
                    other = other->next;
                }
                else if (self->pow > other->pow){
                    result.insert(self->coeff, self->pow);
                    self = self->next;
                }
                else if (self->pow < other->pow){
                    result.insert(other->coeff, other->pow);
                    other = other->next;
                }
            }

            return result;
        }

};

int main(){
    PolynomialLL pol1;
    PolynomialLL pol2;
    PolynomialLL pol3;

    PolynomialLL result;
    cout << "Creating Polynomial 1 by insertion one by one: " << endl;
    pol1.insert(2,1);
    pol1.traverse_display();
    pol1.insert(3,2);
    pol1.traverse_display();
    pol1.insert(3,0);
    pol1.traverse_display();

    cout << "Creating Polynomial 2 by insertion one by one: " << endl;
    pol2.insert(2,1);
```

```
    pol2.traverse_display();
    pol2.insert(3,2);
    pol2.traverse_display();
    pol2.insert(3,0);
    pol2.traverse_display();

    cout << "Creating Polynomial 3 by insertion one by one: " << endl;
    pol3.insert(1,4);
    pol3.traverse_display();
    pol3.insert(5,2);
    pol3.traverse_display();
    pol3.insert(2,3);
    pol3.traverse_display();

    cout << "Adding Polynomial 1 and 2: " << endl;
    result = pol1.add_polynomials(pol2);
    cout << "Result is: " << endl;
    result.traverse_display();

    cout << "Adding result with Polynomial 3: " << endl;
    result = result.add_polynomials(pol3);
    cout << "Result is: " << endl;
    result.traverse_display();
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 05_DoublyLinkedList % ./03_PolynomialLL
Creating Polynomial 1 by insertion one by one:
Current List Status:
|2|1|null|
_____________________________________________________________________

Current List Status:
|3|2|0x10543dd60|------->|2|1|null|
_____________________________________________________________________

Current List Status:
|3|2|0x10543dd60|------->|2|1|0x10543dd80|------->|3|0|null|
_____________________________________________________________________

Creating Polynomial 2 by insertion one by one:
Current List Status:
|2|1|null|
_____________________________________________________________________

Current List Status:
|3|2|0x10543dd90|------->|2|1|null|
_____________________________________________________________________

Current List Status:
```

```
|3|2|0x10543dd90|------->|2|1|0x10543ddb0|------->|3|0|null|
_____________________________________________________________________________

Creating Polynomial 3 by insertion one by one:
Current List Status:
|1|4|null|
_____________________________________________________________________________

Current List Status:
|1|4|0x10543db90|------->|5|2|null|
_____________________________________________________________________________

Current List Status:
|1|4|0x10543dba0|------->|2|3|0x10543db90|------->|5|2|null|
_____________________________________________________________________________

Adding Polynomial 1 and 2:
Result is:
Current List Status:
|6|2|0x10543dbc0|------->|4|1|0x10543dbd0|------->|6|0|null|
_____________________________________________________________________________

Adding result with Polynomial 3:
Result is:
Current List Status:
|1|4|0x10543dbf0|------->|2|3|0x10543dc00|------->|11|2|0x10543dc10|-------
>|4|1|0x10543dc20|------->|6|0|null|
_____________________________________________________________________________
```

Discussions

```
- The polynomial terms are arranged in the list in human readable format,
i.e descending order of exponents.

- Here, we created a separate method called "insert" which takes
coefficient and exponent as argument, finds its appropriate position, and
inserts it in the list.

- We tested two cases, one adding same polynomial to itself, and two,
adding two polynomials with different number of terms and degree.
```

Problem 4: Doubly Linked List Operations.

Code:

```cpp
#include <iostream>
using namespace std;

class node{
```

```
    public:
        int data;
        node* prev;
        node* next;
        node(int value=0, node* _next=nullptr, node* _prev=nullptr){
            this->data = value;
            this->next = _next;
            this->prev = _prev;
        }
        void display(){
            cout << "At address: " << this << endl;
            cout << "Previous Address: " << prev << endl;
            cout << "Data: " << data << endl;
            cout << "Next Address: " << next << endl;
            cout <<
"~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"
<< endl;
        }
};

class LinkedList{
    public:
        node* start;
        LinkedList(node* startptr = NULL){
            this->start = startptr;
        }

        void traverse_display(){
            cout << "Current List Status:" << endl;
            node* cptr = start;
            if (start==nullptr){
                cout << "|null|null|null|" << endl;
            }
            else{
                while (cptr != nullptr){
                    if (cptr->prev == nullptr && cptr->next == nullptr){
                        cout << "|null|" << cptr->data << "|" << "null" <<
"|";
                    }
                    else if (cptr->prev == nullptr){
                        cout << "|null|" << cptr->data << "|" << cptr->next
<< "|" << "------->" ;
                    }
                    else if (cptr->next == nullptr){
                        cout << "|" << cptr->prev << "|" << cptr->data <<
"|" << "null" << "|";
                    }
                    else{
                        cout << "|" << cptr->prev << "|" << cptr->data <<
"|" << cptr->next << "|" << "------->" ;
                    }
                    cptr = cptr->next;
                }
                cout << endl;
```

```
            }
            cout <<
"                                                                                "
<< endl;
            cout << endl;
        }

        bool ins_beginning(int value){
            node* new_ptr = new node;
            if (new_ptr==nullptr) {
                cout << "New Node creation failed. OOM" << endl;
                return 1;
            }
            new_ptr->data = value;
            if (start == nullptr){
                start = new_ptr;
            }
            else{
                start -> prev = new_ptr;
                new_ptr->next = start;
                start = new_ptr;
            }
            cout << "Successfully added a new node at beginning! The new
node added is:" << endl;
            new_ptr->display();
            return 0;
        }
        bool ins_end(int value){
            node* new_ptr = new node(value);
            if (new_ptr==nullptr) {
                cout << "New Node creation failed. OOM" << endl;
                return 1;
            }
            if (start == nullptr){
                start = new_ptr;
            }
            else{
                node* ptr = start;
                while (ptr->next!=nullptr){
                    ptr = ptr->next;
                }
                new_ptr->prev = ptr;
                ptr->next = new_ptr;
            }
            cout << "Successfully added a new node at end! The new node
added is:" << endl;
            new_ptr->display();
            return 0;
        }
        bool ins_after_specific(int value, int target){
            node* new_ptr = new node;
            if (new_ptr==nullptr) {
                cout << "New Node creation failed. OOM" << endl;
                return 1;
```

```
            }
            new_ptr->data = value;
                node* ptr = start;
                while (ptr!=nullptr && ptr->data!=target){
                    ptr = ptr->next;
                }
                if (ptr==nullptr) {
                    cout << "Target node not found!!" << endl;
                    return 1;
                }
                node* postptr = ptr->next;
                if (postptr != nullptr){
                    postptr->prev = new_ptr;
                    new_ptr->next = postptr;
                }
                    new_ptr->prev = ptr;
                    ptr->next = new_ptr;
            cout << "Successfully added a new node after specified node: "
<< target<<" ! The new node added is:" << endl;
            new_ptr->display();
            return 0;
        }
        bool ins_before_specific(int value, int target){
            node* new_ptr = new node;
            if (new_ptr==nullptr) {
                cout << "New Node creation failed. OOM" << endl;
                return 1;
            }
            new_ptr->data = value;
            if (start == nullptr){
                start = new_ptr;
            }
            else{
                node* postptr = start;
                node* preptr = start;
                while (postptr!=nullptr && postptr->data!=target){
                    preptr = postptr;
                    postptr = postptr->next;
                }
                if (postptr==nullptr) {
                    cout << "Target node not found!!" << endl;
                    return 1;
                }
                postptr->prev = new_ptr;
                new_ptr->next = postptr;
                new_ptr->prev = preptr;
                preptr->next = new_ptr;
            }
            cout << "Successfully added a new node before specified node: "
<< target<<" ! The new node added is:" << endl;
            new_ptr->display();
            return 0;
        }
        bool del_beginning(){
```

```
            if (start==nullptr){
                cout << "The list is empty: Underflow!!" << endl;
                return 1;
            }
            else{
                node* temp = start;
                node* posttemp = start->next;
                if (posttemp!=nullptr) posttemp->prev = nullptr;

                start = posttemp;
                cout << "Successfully deleted the node at beginning. The
deleted node was:" << endl;
                temp->display();
                delete temp;
            }
            return 0;
        }
        bool del_end(){
            if (start==nullptr){
                cout << "The list is empty: Underflow!!" << endl;
                return 1;
            }
            else{
                node* temp = start;
                node* pretemp = start;
                while (temp->next != nullptr){
                    pretemp = temp;
                    temp = temp->next;
                }
                pretemp->next = nullptr;
                cout << "Successfully deleted the node at end. The deleted
node was:" << endl;
                temp->display();
                delete temp;
            }
            return 0;
        }
        bool del_after_specific(int target){
            if (start==nullptr){
                cout << "The list is empty: Underflow!!" << endl;
                return 1;
            }
            else{
                node* pretemp = start;
                while (pretemp!=nullptr && pretemp->data != target){
                    pretemp = pretemp->next;
                }
                if (pretemp==nullptr){
                    cout << "Node "<< target <<" not found!!" << endl;
                    return 1;
                }
                if (pretemp->next != nullptr){
                    node* temp = pretemp->next;
                    node* posttemp = temp->next;
```

```
                    if (posttemp != nullptr) posttemp->prev = pretemp;
                    pretemp->next = posttemp;
                    cout << "Successfully deleted after specified node: "<<
target <<" . The deleted node was:" << endl;
                    temp->display();
                    delete temp;
                }
                else{
                    cout << "No node after "<< target <<"! What shall I
delete?" << endl;
                    return 1;
                }
            }
            return 0;
        }

         bool del_before_specific(int target){
            if (start==nullptr){
                cout << "The list is empty: Underflow!!" << endl;
                return 1;
            }
            else{
                node* posttemp = start;
                while (posttemp!=nullptr && posttemp->data != target){
                    posttemp = posttemp->next;
                }
                if (posttemp==nullptr){
                    cout << "Node "<< target <<" not found!!" << endl;
                    return 1;
                }
                    node* temp = posttemp->prev;
                    node* pretemp = temp->prev;
                    posttemp->prev = pretemp;
                    pretemp->next = posttemp;
                    cout << "Successfully deleted before specified node: "
<< target <<" . The deleted node was:" << endl;
                    temp->display();
                    delete temp;
                }
            return 0;
            }
};

int main(){

    LinkedList myLL;

    myLL.traverse_display();

    cout << "Insertion at Beginning: " << endl;
    myLL.ins_beginning(2);
    myLL.traverse_display();

    cout << "Insertion at End: " << endl;
```

```
    myLL.ins_end(3);
    myLL.traverse_display();

    cout << "Insertion before element 3: " << endl;
    myLL.ins_before_specific(4, 3);
    myLL.traverse_display();

    cout << "Insertion after element 3: " << endl;
    myLL.ins_after_specific(5, 3);
    myLL.traverse_display();

    cout << "Deletion at Beginning: " << endl;
    myLL.del_beginning();
    myLL.traverse_display();

    cout << "Deletion at End: " << endl;
    myLL.del_end();
    myLL.traverse_display();

    cout << "Deletion after 4: " << endl;
    myLL.del_after_specific(4);
    myLL.traverse_display();

    cout << "Deletion after 20: " << endl;
    myLL.del_after_specific(20);
    myLL.traverse_display();

    cout << "Deletion at Beginning: " << endl;
    myLL.del_beginning();
    myLL.traverse_display();

    cout << "Deletion at End: " << endl;
    myLL.del_end();
    myLL.traverse_display();

    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 05_DoublyLinkedList % ./04_DoublyLL
Current List Status:
|null|null|null|
_____________________________________________________________________

Insertion at Beginning:
Successfully added a new node at beginning! The new node added is:
At address: 0x101335da0
Previous Address: 0
Data: 2
Next Address: 0
```

```
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|2|null|
_____________________________________________________________________________

Insertion at End:
Successfully added a new node at end! The new node added is:
At address: 0x101335dc0
Previous Address: 0x101335da0
Data: 3
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|2|0x101335dc0|------->|0x101335da0|3|null|
_____________________________________________________________________________

Insertion before element 3:
Successfully added a new node before specified node: 3 ! The new node added
is:
At address: 0x101335de0
Previous Address: 0x101335da0
Data: 4
Next Address: 0x101335dc0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|2|0x101335de0|------->|0x101335da0|4|0x101335dc0|-------
>|0x101335de0|3|null|
_____________________________________________________________________________

Insertion after element 3:
Successfully added a new node after specified node: 3 ! The new node added
is:
At address: 0x101335bc0
Previous Address: 0x101335dc0
Data: 5
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|2|0x101335de0|------->|0x101335da0|4|0x101335dc0|-------
>|0x101335de0|3|0x101335bc0|------->|0x101335dc0|5|null|
_____________________________________________________________________________

Deletion at Beginning:
Successfully deleted the node at beginning. The deleted node was:
At address: 0x101335da0
Previous Address: 0
Data: 2
Next Address: 0x101335de0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|4|0x101335dc0|------->|0x101335de0|3|0x101335bc0|-------
>|0x101335dc0|5|null|
_____________________________________________________________________________
```

```
Deletion at End:
Successfully deleted the node at end. The deleted node was:
At address: 0x101335bc0
Previous Address: 0x101335dc0
Data: 5
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|4|0x101335dc0|------->|0x101335de0|3|null|
_____________________________________________________________________________

Deletion after 4:
Successfully deleted after specified node: 4 . The deleted node was:
At address: 0x101335dc0
Previous Address: 0x101335de0
Data: 3
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|4|null|
_____________________________________________________________________________

Deletion after 20:
Node 20 not found!!
Current List Status:
|null|4|null|
_____________________________________________________________________________

Deletion at Beginning:
Successfully deleted the node at beginning. The deleted node was:
At address: 0x101335de0
Previous Address: 0
Data: 4
Next Address: 0
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Current List Status:
|null|null|null|
_____________________________________________________________________________

Deletion at End:
The list is empty: Underflow!!
Current List Status:
|null|null|null|
_____________________________________________________________________________
```

Discussions

- During insertion, preptr and ptr are the pointers that point to nodes surrounding the newnode after insertions, and during deletion, pretemp and posttemp bound the temp node which is to be deleted.

- We followed the standard algorithms for insertion and deletion operations, without the use of end pointer.

- We also created methods that display the new node or deleted node, and method that traverses the list and displays the entire list in a readable format.

- The size of each node object is exactly 16 bytes (int+padding+pointer : 4+8+8=16bytes)

- It is clear that every new operation allocates memory contigiously from heap. And new allocations skip the address by exactly the node size bytes

- However, we scramble this order of address in heap by adding and deleting nodes randomly within the list.

- Thus, nodes in a doubly linked list are scattered randomly in heap. This makes the data structure highly flexible but also has disadvantages such as cache unfriendliness, no random access (needs traversal), memory fragmentation issues etc.

- Doubly Linked Lists have the advantage of allowing traversal in opposite direction (end to start), making certain operations faster.

Conclusion

At the end of the lab, we can conclude that we have sucessfully studied and understood applications of singly linked lists and implementation of doubly linked lists, both theoretically and practically.